
Documentation: file_monitor.py

Overall Purpose: The File System Watcher

This module's job is to be the EDR's eyes on the filesystem. It watches specific files and folders defined in config.json and reports any activity—such as files being created, deleted, modified, or renamed.

This is a "specialist worker" built from the BaseMonitor blueprint. The line class FileMonitor(BaseMonitor): shows that it **inherits** all the standard features from BaseMonitor, like the ability to log alerts and a standard lifecycle.

A key feature of this monitor is that it also performs **content scanning**. When a file is created or changed, it peeks inside the file to see if it contains anything suspicious, like patterns matching web attacks defined in the config.

The Magic Ingredient: The watchdog Library

Instead of constantly re-checking files every few seconds to see if they've changed (which is very inefficient), this monitor uses a powerful Python library called watchdog.

How it works: watchdog asks the operating system to notify it whenever a change happens in a specific directory. This is an **event-based** approach, which is extremely efficient because our program uses almost no CPU while waiting for an event to occur. The watchdog library runs its own little process in the background (a **background thread**) to listen for these notifications.

Code Breakdown

1. The FileMonitor Class

This is the main class that plugs into the EDR agent framework. It's responsible for setting up and managing the watchdog observer.

`get_name(self)`

- This is the first rule of the BaseMonitor blueprint being fulfilled.
- It returns the string "file_integrity". The EDRAgent uses this name to find the monitor's specific settings in the config.json file.

`__init__(self, ...)` (The Constructor)

- `super().__init__(...)`: This is a critical first step. It calls the `__init__` method of its parent, BaseMonitor, to ensure all the standard setup (like storing the config and log queue) is done.
- `self.observer = None`: It prepares a placeholder for the watchdog object that will be created later.
- `self.web_attack_patterns = [...]`: This line reads the `web_attack_patterns` from the configuration.
 - `re.compile(...)`: For each pattern, it "compiles" it using Python's regular expression library (`re`). Compiling a pattern beforehand is an optimization that makes searching for it inside files much faster.

`start(self)`

- This method brings the monitor to life. It overrides the empty start method from the blueprint.

- It creates an Observer object from watchdog. The observer is the component that will run in the background.
- It creates an instance of our own `_FileChangeHandler` class. This handler is the component that defines *what to do* when the observer notices a change.
- It reads the list of paths from its configuration.
- For each path, it tells the observer to watch it using the handler:
`self.observer.schedule(...)`.
- Finally, `self.observer.start()` launches the observer in a new background thread. From this moment on, the file monitoring is active.

`stop(self)`

- This is the cleanup method. When the main agent signals a shutdown, this method is called.
- It safely stops the watchdog observer thread (`self.observer.stop()`) and waits for it to finish completely (`self.observer.join()`).

`run(self)`

- **This method is intentionally left empty.** The BaseMonitor's `run_wrapper` calls this method on a loop every few seconds. However, since all the real work is being done by the watchdog library in a separate background thread, our main monitor process has nothing to do. It can just sit idle while watchdog handles everything. This is a very efficient design.

2. The `_FileChangeHandler` Class

This is a helper class designed to work directly with watchdog. The underscore `_` at the beginning of its name is a Python convention that signals it's an "internal" part of the file monitor module.

Its job is to receive events from the Observer and translate them into alerts for our EDR agent.

`on_created(self, event), on_deleted(self, event), on_modified(self, event), on_moved(self, event)`

- These methods are special names that watchdog automatically calls when the corresponding event happens.
- For example, when a new file is created in a monitored directory, watchdog calls the `on_created` method.
- Each method takes the event information and uses the monitor's `log_alert()` function to send a structured message back to the main agent.
- **Security Check:** Notice that `on_created` and `on_modified` both call `_scan_file_content`. This is a key security feature—they check the file for malicious content the moment it appears or changes.

`_scan_file_content(self, file_path)`

- This is the core detection logic for this monitor.
- It attempts to open the file specified by `file_path` and read its contents. (It only reads the first 1MB to avoid performance issues with very large files).
- It then loops through all the pre-compiled `suspicious_patterns`.
- `pattern.search(content)`: This uses the powerful regular expression engine to search for a match.
- If a match is found, it immediately logs a high-priority alert and returns `True`.
- The `try...except` block provides resilience. If the file can't be opened (e.g., it was deleted instantly, or we don't have the right permissions), the monitor won't crash; it will simply skip the scan.